

LC 1584: MIN COST TO CONNECT ALL POINTS

Target Companies: Chegg, Deshow, TCS

Problem Understanding

basically X and Y coordinates are given , dist bw each is manhattan distance , need to connect all point in min cost , cost == distance

whenever a question says connect all nodes and get minimum cost where there is no cycle, think of MST

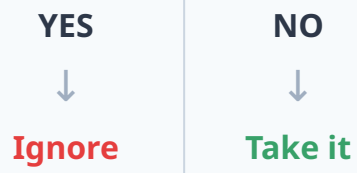
there are only 2 MST algorithms:

- (I) Kruskals (DSU)
- (II) Prims (priority queue)

Kruskal Algorithm Overview



(iv) Cycle Check



Why ignore cycles?

Cycle are ignored because we need minimum cost and a cycle doesn't help to reach any new node.

(v) repeat for all edges

C++ Implementation Logic

Explanation & Approach:

- we need to store $\{\text{dist}, \{i, j\}\}$ for every possible edge.
- baad me sort kr denge low to high.
- basically kruskal.
- ab ek ek kar ke edges process krenge.
- agar koi edge cycle bana rha he to its useless , bas bojh he jo cost badhayega. doesnt create new connectivity , isliye cycle ignore.
- otherwise uska cost ans me add.
- jaise hi n-1 edge ho jaayenge kaam khtm.

Source Code

```
class dsu {
public:
    vector<int> parent;
    vector<int> size;

    dsu(int n) {
        parent.resize(n);
        size.resize(n, 1);
        for(int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    int findpar(int node) {
        if(parent[node] == node) return node;
        parent[node] = findpar(parent[node]);
        return parent[node];
    }

    void merge(int u, int v) {
        int pu = findpar(u);
```

```

    int pv = findpar(v);

    if(pu == pv) return;

    if(size[pu] < size[pv]) {
        swap(pu, pv);
    }

    size[pu] += size[pv];
    parent[pv] = pu;
}

};

class Solution {
public:
    int minCostConnectPoints(vector<vector<int>>& points) {
        int n = points.size();
        dsu ds(n);

        // we need to store {dist,{i,j}} for every possible edge
        // baad me sort kr denge low to high
        // basically kruskal

        vector<pair<int, pair<int, int>>> edges;
        for(int i = 0; i < n; i++) {
            for(int j = i + 1; j < n; j++) {
                int dist = abs(points[i][0] - points[j][0]) + abs(points[i][1] - points[j][1]);
                edges.push_back({dist, {i, j}});
            }
        }

        sort(edges.begin(), edges.end());

        // ab ek ek kar ke edges process kreng
        // agar koi edge cycle bana rha he to its useless , bas bojhe jo
        cost badhayega
        // doesnt create new connectivity , isliye cycle ignore
        // otherwise uska cost ans me add
        // jaise hi n-1 edge ho jaayenge kaam khtm

        int ans = 0;
        int edgecnt = 0;

        for(auto &it : edges) {
            int dist = it.first;
            int u = it.second.first;
            int v = it.second.second;

```

```
    if(ds.findpar(u) == ds.findpar(v)) continue;

    ds.merge(u, v);
    ans += dist;
    edgecnt++;

    if(edgecnt == n - 1) break;
}

return ans;
}

};
```